Semantic State Resolution (SSR)

Architectural Standard for Just-In-Time Semantic Reality

Author: Patrick Richardson
Date: June 19, 2026
Status: Architectural Standard v1.1

0. Component Framing

- Semantic State Resolution (SSR) is a domain-general architecture for resolving latent, partially known, or semantically ambiguous state into canonical, replayable, provenance-backed state.
- SSR is not limited to games, simulations, or narrative engines. Those are valid domain profiles, not the boundary of the concept.
- SSR supports canonical state, event-sourced persistence, latent entity collapse, property-level collapse, omnidimensional neighbor resolution, constraint/provenance tracking, and no-retcon behavior.
- Symbolic application logic remains the final authority for hard rules. The LLM proposes, interprets, and explains; it does not directly mutate canonical state.
- Domain-specific systems may layer their own rule kernels, validation tables, schemas, and profiles on top of SSR Core.

1. Executive Summary

Semantic State Resolution (SSR) is an architecture for systems that must operate over incomplete reality without allowing hallucinated, silent, or contradictory state mutation.

Traditional software often treats structured state and semantic meaning as separate domains. Databases hold fields; prose, labels, notes, generated descriptions, and interpreted meaning live outside the authoritative state model. That separation causes semantic drift: the system can describe a thing one way while behaving as if that description has no operational consequence.

SSR resolves this by treating semantic meaning, structured properties, constraints, and hard rules as different authority layers over the same canonical state. Reality may begin latent. It becomes explicit only when observation, action, decision, validation, or downstream dependency requires it.

Unlike pure generative AI, SSR does not grant the model authority over truth. The model is a proposer/interpreter. The symbolic engine validates proposals against schemas, whitelists, authority boundaries, rule kernels, provenance, and canonical consistency before anything commits.

SSR is no-retcon: every accepted observation, action, interpretation, time advance, system event, property collapse, or entity collapse commits to an event log before it affects the canonical projection.

2. Problem Statement: The Three Traps

Procedural and AI-assisted systems suffer from three recurring failures that SSR is designed to prevent.

2.1 The Semantic Gap

Standard software state is often syntactically valid but semantically blind.

The Failure: A workflow item, record, agent memory, case file, generated asset, knowledge claim, or system object may have valid fields while contradicting context, provenance, user-visible meaning, or downstream constraints.

Example: A generated recommendation claims a customer is high priority, but the canonical record contains unresolved eligibility, obsolete contact data, and conflicting source evidence. The prose appears coherent; the state is not safely actionable.

2.2 The Hallucination Trap

Pure generative systems lack state rigidity. Without canonical anchors, models suffer from object impermanence.

The Failure: A fact described as tentative in one step may become certain in a later step. A rejected claim may reappear as accepted. A source may be forgotten. A hidden assumption may silently become operational state.

Result: The application behaves like a dream: locally responsive, but not causally durable.

2.3 The Latency Trap

To maintain coherence, many systems either pre-resolve too much state or defer too much state to unstructured generation.

The Failure: A system may fully instantiate records, dependencies, classifications, or derived properties long before they are needed, wasting compute and forcing premature commitments. Alternatively, it may leave too much to runtime prose and lose deterministic replay.

Result: The system becomes either over-materialized and rigid, or under-constrained and unreliable.

3. Core Definitions & Invariants

3.1 Latent Entity

A latent entity is a stable canonical handle with unresolved properties. It exists as an ID, provenance, constraints, and indexed property handles, but it does not need a fully realized component set.

3.2 Collapsible Indexed Property

A collapsible indexed property is a first-class state unit attached to an entity or domain object. It may be latent, proposing, collapsed, rejected, obsolete, or canonical.

Entities are therefore not fixed bags of hard-coded fields. An entity is an indexed bundle of property handles whose values may resolve progressively under observation and validation.

Hard-coded structure is still required for the SSR kernel: property IDs, namespaces, lifecycle states, authority levels, validators, visibility scopes, provenance fields, schemas, whitelists, and event contracts. What becomes collapsible is the property value, the property existence where allowed, the semantic implication, the visible form, or the domain-specific affordance.

3.3 Omnidimensional Neighbor

An omnidimensional neighbor is any typed, scoped, weighted source of constraint pressure that may legitimately influence the resolution of an entity or property.

A neighbor is not only a spatially adjacent object or graph-adjacent node. A neighbor may be spatial, temporal, causal, semantic, mechanical, social, epistemic, provenance-based, affordance-based, or narrative.

The purpose of omnidimensional neighbors is not to broaden LLM authority. Neighbor influence must feed the constraint graph and validation layer. Canonical state, hard rules, and event provenance remain superior to model plausibility.

3.4 Observation

An observation is the event where a user, system, rule, API request, dependency, or internal process queries a latent entity or property strongly enough to require resolution or projection.

Observation does not always collapse everything. It collapses only the minimum property set required for the current authority boundary and response contract.

3.5 Observation Scope

Observation scope is defined by the interaction horizon. Depending on the domain, this may include:

- **Visual Horizon:** what a user interface, viewer, camera, or inspection surface may expose.
- **Logical Horizon:** graph distance, dependency reachability, linked records, or adjacent workflow nodes.
- **Semantic Horizon:** properties, tags, claims, sources, or meanings close enough to constrain resolution.
- **Authority Horizon:** the subset of state the current actor or subsystem is allowed to observe or commit.

3.6 Canonical State

Canonical state is a projection derived from the event log. It is the fast runtime view of truth, but the event log remains the replayable source of truth.

3.7 The No-Retcon Invariant

Once an entity or property resolves into canonical state, its realized value may only change through recorded deltas caused by valid system events, user actions, rule outcomes, or correction workflows. Later model output may not silently overwrite it.

Unobserved entities and unresolved properties remain latent, defined by constraints and provenance.

4. Architecture & Data Contracts

4.1 System Architecture

SSR uses the application engine as a mediator between latent state, symbolic rules, constraint storage, proposal systems, validation, event sourcing, and canonical projection.

The model may propose. The engine validates. The event log records. The projection serves.

4.2 Constraint Store

Unresolved reality is defined by a constraint graph, not by unrestricted generation.

Each constraint is stored as a record:

```
{ "key": "...", "value": "...", "strength": 0.5, "type":
"hard|soft", "source_event_id": "...", "ttl": 10 }
```

Pruning Policy:

- Soft constraints are dropped when strength falls below the configured threshold or when TTL expires.
- Hard constraints persist unless explicitly terminated by a recorded event such as `ConstraintObsoleted`, `ConstraintContradicted`, or a domain-specific correction event.

4.3 Property Index

SSR should maintain a property index so entities can resolve progressively.

```
type PropertyState = "latent" | "proposing" | "collapsed" |
"rejected" | "obsolete" | "canonical";

type PropertyAuthority =
  | "kernel"
  | "canonical"
  | "hard_constraint"
  | "soft_constraint"
  | "rumor"
  | "belief"
  | "proposal";
```

```
interface SSRProperty {
  property_id: string;
  entity_id: string;
  key: string;
  namespace:
    | "mechanic"
    | "semantic"
    | "sensory"
    | "social"
    | "spatial"
    | "temporal"
    | "narrative"
    | "affordance"
    | "provenance";
  state: PropertyState;
  authority: PropertyAuthority;
  value?: unknown;
  validator_id?: string;
  whitelist_id?: string;
  visibility_scope?: string;
  source_event_id: string;
}
```

Property-level collapse prevents premature commitment. An entity may be canonical as a handle while specific properties remain unresolved.

4.4 Omnidimensional Neighbor Index

SSR should maintain typed neighbor edges so resolution can use all admissible constraint pressure without confusing all relations as spatial adjacency.

```
type NeighborDimension =
  | "spatial"
  | "temporal"
  | "causal"
  | "semantic"
  | "mechanical"
  | "social"
  | "epistemic"
  | "provenance"
```

```
  | "affordance"
  | "narrative";

interface OmnidimensionalNeighborEdge {
  id: string;
  target_ref: { kind: "entity" | "property"; id: string };
  source_ref: { kind: string; id: string };
  dimension: NeighborDimension;
  relation: string;
  authority: PropertyAuthority;
  strength: number;
  ttl?: number;
  source_event_id: string;
}
```

Neighbor edges may influence proposal context only through deterministic selection, recorded provenance, and validation.

4.5 Solver Interface

The boundary between the application engine and a model proposer is strict.

1. **Engine Request:** Sends context, selected constraints, selected neighbor edges, schema, allowed IDs, and authority scope.
2. **LLM Proposal:** Returns structured JSON proposals.
3. **Engine Validation:** Checks schema, ranges, whitelists, rule compatibility, authority boundaries, and canonical consistency.
4. **Commit:** Validated data is written to the event log.

4.6 Deterministic Fallback Protocol

If the model fails to produce a valid proposal within the configured retry budget, the system executes a deterministic fallback.

Selection Mechanism:

```
Index = Hash(Target_ID + Hard_Constraints + Ruleset_Version) %
Safety_Table_Size
```

Invariant: The application loop must never block or crash because a proposal system failed.

4.7 Persistence Model

SSR relies on event sourcing for replayability and consistency. The event log is the source of truth; the canonical projection is its current materialized view.

Representative event types:

- `EntityRegistered`
- `PropertyHandleCreated`
- `PropertyResolutionCommitted`
- `ResolutionCommitted`
- `DeltaApplied`
- `ConstraintInjected`
- `ConstraintObsoleted`
- `ProposalRejected`
- `NeighborEdgeCreated`
- `NeighborEdgeExpired`

5. The SSR Lifecycle

Phase 1: Latent State

The system contains a stable entity handle and one or more latent properties. The handle is canonical; the unresolved values are not.

Phase 2: Observation or Dependency Trigger

A user request, system action, rule dependency, query, projection, or downstream operation requires some property or entity state to become explicit.

Phase 3: Neighborhood and Constraint Selection

The engine gathers admissible constraints from canonical state, hard rules, property indexes, and omnidimensional neighbors.

Phase 4: Proposal

The engine sends a bounded request to the proposer. The request includes schema, whitelists, selected constraints, selected neighbor context, and authority scope.

Phase 5: Validation & Resolution

The engine validates the proposal. Valid results commit as events and update canonical projections. Invalid proposals are rejected with recorded reasons.

Phase 6: Propagation

Resolved entities or properties may inject new constraints into other unresolved entities, properties, or neighbor edges. Propagation is event-backed and provenance-tracked.

Note on Latency & Future Proofing

SSR is latency agnostic. Lookahead buffers may pre-resolve likely dependencies, but the architecture does not require total pre-simulation. As inference and validation costs decrease, buffer size can shrink without changing the core invariants.

6. Conflict Resolution & Precedence

When constraints conflict, resolution follows this hierarchy:

1. **Canonical State:** Already committed state is authoritative.
2. **Kernel Rules:** Domain rule kernels and validation contracts cannot be overridden by prose.
3. **Hard Constraints:** Requirements created by prior events or authority-bearing systems.
4. **Soft Constraints:** Themes, tendencies, weak signals, heuristics, and non-binding context.
5. **LLM Generation:** Creative filler, interpretation, and proposal content.

If a hard constraint violates canonical state, the system records the conflict and marks the constraint obsolete, contradicted, or requiring correction. It does not retcon canonical state silently.

7. Domain-General Application Profiles

SSR Core is domain-general. A domain profile supplies schemas, validators, whitelists, authority rules, and projection formats.

Possible profiles include:

- **Knowledge Management:** claims, evidence, source authority, contradictions, obsolete facts, and provenance-backed conclusions.

- **Case Management:** intake facts, eligibility criteria, documents, determinations, follow-up actions, and audit trails.
- **Enterprise Workflow:** tasks, owners, blockers, dependencies, risk states, approvals, and operational decisions.
- **AI Agent Systems:** interpreted user intent, proposed actions, tool results, memory records, and committed plans.
- **Compliance and Audit:** obligations, exceptions, control evidence, policy mappings, findings, and remediation state.
- **Interactive Simulation:** generated environments, actors, items, events, and player-facing projections as one possible profile rather than the core scope.

8. Unified Thesis

SSR should not resolve fixed entities in isolation.

SSR should resolve indexed properties of stable entity handles under admissible constraint pressure from omnidimensional neighbors, while preserving canonical authority, validation, event sourcing, provenance, and no-retcon behavior.

The corrected architectural thesis is:

Semantic State Resolution is a domain-general architecture for converting latent, partially known, or semantically ambiguous state into canonical, replayable, provenance-backed state through observation-triggered resolution, constrained proposal, validation, and event-sourced commitment.

9. Appendix: Reference Data Contracts

9.1 Solver Request

```
{
  "request_id": "req_8821a",
  "task_type": "RESOLVE_PROPERTY",
  "target": {
    "kind": "property",
    "entity_id": "record_104",
    "property_id": "prop_104_status"
  },
  "context": {
```

```
    "canonical_summary": {},
    "omnidimensional_neighbors": {
      "temporal": [],
      "causal": [],
      "semantic": [],
      "mechanical": [],
      "social": [],
      "epistemic": [],
      "provenance": [],
      "affordance": [],
      "narrative": []
    }
  },
  "constraints": {
    "hard": [],
    "soft": []
  },
  "schema": {
    "$schema": "https://json-schema.org/draft/2020-12/schema",
    "type": "object",
    "required": ["property_id", "proposed_value"],
    "properties": {
      "property_id": { "type": "string" },
      "proposed_value": { "type": ["string", "number",
"boolean", "object", "array", "null"] },
      "tags": {
        "type": "array",
        "items": { "type": "string" }
      },
      "explanation": { "type": "string" }
    },
    "additionalProperties": false
  },
  "authority_scope": {
    "requesting_actor": "kernel.resolution_engine",
    "requesting_subsystem": "proposal_pipeline",
    "visibility_scope": ["resolver", "auditor",
"authorized_observer"],
    "proposal_authority": "proposal",
    "context_authority_levels": ["kernel", "hard_constraint",
```

```
"soft_constraint"]
  },
  "whitelist": {
    "status_ids": ["unknown", "pending", "validated",
"rejected", "obsolete"],
    "action_ids": ["request_evidence", "mark_conflict",
"commit_status"],
    "tag_ids": ["needs_evidence"]
  }
}
```

9.2 Solver Proposal

```
{
  "request_id": "req_8821a",
  "proposal": {
    "property_id": "prop_104_status",
    "proposed_value": "pending",
    "tags": ["needs_evidence"],
    "explanation": "Existing source evidence is insufficient for
validated status."
  }
}
```

9.3 Validator Response

```
{
  "status": "PASS",
  "validated_payload": {
    "property_id": "prop_104_status",
    "canonical_value": "pending",
    "events": ["PropertyResolutionCommitted",
"ConstraintInjected"]
  }
}
```